산으로 가는 AI 조련법

Upstage Workshop — CC를 제대로 부리는 기술

2026.02.25

오늘의 구조

시간	파트	핵심
1시간	조련법	Clarify 렌즈 + 멀티 프로토타이핑 + Wrap/Compound
2시간	해커톤	직접 문제 해결 — **more token, more wins**
1시간	발표	결과물 시연 + 피드백

> *"배웠습니다"가 아니라 **"만들었습니다"***
> *CLAUDE.md + Skill = **지식이 복리로 쌓이는 구조***

1. 산으로 가는 AI

이미 겪어봤을 이야기

"간단한 작업은 되는데

복잡한 건 산으로 간다"

산으로 가는 3가지 패턴

패턴 1: 모호한 지시

"헬스체크 추가해줘" → DB 체크? 포맷? 경로? **CC가 추측 → 틀림**

패턴 2: Unknown Unknown이 뒤늦게 드러남

구현 다 되고 나서야 **암묵지였던 영역**을 발견. 내가 뭘 모르는지 몰랐음.

패턴 3: 복리로 쌓지 못함

정보는 쌓여도 **insight가 쌓이지 않음**. 매번 처음부터 시작.

근본 원인 → 해결 매핑

```
의도 ── Gap 1 ─── CC 해석 ── Gap 2 ── 실행 결과
                                        |
                                   Gap 3: 미축적
```

Gap	원인	해결	방법
Gap 1	모호한 요구사항	**Clarify**	3가지 렌즈
Gap 2	숨은 가정, 암묵지	**멀티 프로토타이핑**	Worktree N개
Gap 1→2	계획 없이 실행	**Plan + Implement**	배치 실행
Gap 3	인사이트 미축적	**Wrap + Compound**	5-agent + 지식 축적

2. 조련법: Clarify

3가지 렌즈로 의도를 끄집어내기

Clarify = 3가지 렌즈

단순히 "질문 4개"가 아니다. **내 의도를 끄집어내서 최대한 명확하게** 만드는 것.

렌즈	스킬	해결하는 것
Vague	`/vague`	모호한 요구사항 → 구체적 스펙
Unknown	`/unknown`	숨은 가정 → Known/Unknown 4분면
Metamedium	`/metamedium`	내용 vs 형식 → 진짜 레버리지

> *상황에 따라 **적합한 렌즈**를 선택해서 의도를 명확화*

/vague — 모호함을 옵션으로

원리: 열린 질문 대신 **가설을 옵션으로** 제시

```
BAD:  "어떤 인증을 원해?"              ← 열린 질문, 인지 부하 높음
GOOD: "JWT / API Key / SSO / Magic link"  ← 고르기만 하면 됨
```

프로토콜:

1. 원래 요구사항 그대로 기록
2. 모호한 부분 식별 (뭐가 불명확한가?)
3. **가설-옵션 형태**로 5~8개 질문
4. Before/After 정리

> *"알아서 해줘" → CC가 추측 → 산으로 감*
> *"옵션 중에 골라줘" → CC가 정확히 실행*

/unknown — 숨은 가정 발견

원리: Known/Unknown **4분면**으로 맹점 탐색

```
                 알고 있다          모르고 있다
              ┌──────────────────┬──────────────────┐
알고          │ Known Known      │ Known Unknown    │
있다          │ 확정된 사실      │ 조사 필요        │
              ├──────────────────┼──────────────────┤
모르고        │Unknown Known     │Unknown Unknown   │
있다          │ 암묵지           │ 맹점 ← 위험!     │
              └──────────────────┴──────────────────┘
```

3라운드 심층 질문: R1(전체 탐색) → R2(약점 집중) → R3(실행 디테일)

> *"내가 뭘 모르는지 모르겠어" → 이 스킬이 정확히 그 상황*

/metamedium — 형식을 바꿔라

원리: 대부분 **내용(content)**만 바꾸지만, 진짜 레버리지는 **형식(form)** 변경

	Content (what)	Form (how)
예시	배포 체크 수동 실행	**Skill로 만들어서 1분에 끝냄**
예시	PR 리뷰 코멘트 작성	**리뷰 에이전트가 자동 분석**
레버리지	선형 — 매번 1회	**지수적 — 한 번 만들면 무한 반복**

> *"같은 방식이 안 먹혀" → **형식 자체를 바꿔볼 때***

3. 조련법: 멀티 프로토타이핑

Clarify만으로 Unknown Unknown은 100% 못 잡는다

Clarify 한계 → 빠른 프로토타입으로 보완

Clarify로 모호함을 줄여도, **Unknown Unknown**은 만들어봐야 드러난다.

```
Step 1: clarify로 요구사항 정제

Step 2: ASCII로 N개 설계안 비교
   ┌ A: 미들웨어 체인 ┐
   ├ B: 데코레이터     ┤ 비교 → 문제점 발견
   └ C: 프록시 패턴   ┘

Step 3: Worktree에서 간단한 구현 → Unknown 발견!
```

```
# Worktree 2개 생성 → 각각 CC 실행
git worktree add .worktrees/proto-a -b proto/middleware
git worktree add .worktrees/proto-b -b proto/decorator
cd .worktrees/proto-a && claude "미들웨어 체인으로 JWT 구현"
cd .worktrees/proto-b && claude "데코레이터로 JWT 구현"
# → "미들웨어는 /health도 걸리네" ← Unknown 발견!
```

> ***핵심**: 프로토타입의 목적은 완성이 아니라 **Unknown 발견***

4. 조련법: Plan + Implement

계획 → 배치 실행 → 안전한 전달

/plan → /implement

/plan (`Shift+Tab` x2 → Plan Mode)

- Edit/Write **차단** → 코드 수정 없이 계획만
- 계획이 **파일로 저장** → Git 추적
- 각 Task 2~5분 단위, **1Task = 1커밋**

/implement

- **3개씩** 배치 실행 → 확인 → 다음 배치
- 에러 시 **추측 대신 즉시 보고**

```
[T1, T2, T3] → 실행 → 확인 → [T4, T5, T6] → 실행 → ...
```

> ***credential 관리 팁***: `.claude/settings.local.json` *에 env 변수 설정*
> *AWS_PROFILE 등 민감 정보는* `.env` *보다* ***시스템 환경변수*** *권장*

5. 조련법: Wrap + Compound

인사이트를 복리로 쌓기

/wrap — 5개 에이전트 병렬 분석

에이전트	역할
doc-updater	**CLAUDE.md** 업데이트 제안
automation-scout	**반복 업무 → Skill 변환 제안**
learning-extractor	배운 것, 실수, 발견 추출
duplicate-checker	중복 제거 (Phase 2)
followup-suggester	다음 할 일 우선순위

일 → Skill 변환 루프:

```
wrap → automation-scout: "배포 체크를 Skill로 변환 가능"
     → SKILL.md 생성 → /deploy-check 실행 → 30분 → 1분
```

> ***CLAUDE.md**에 쌓이는 것 = 프로젝트의 **암묵지가 명시지로** 전환되는 것*

/compound — 인사이트를 지식 베이스로

문제: 정보는 쌓여도 insight가 쌓이지 않음

해결: 검증된 인사이트를 **구조화된 문서**로 즉시 축적

```
"이 방식이 잘 됐다" → 트리거 감지
     ↓
context 수집 → YAML frontmatter 문서 생성
     ↓
knowledge/solutions/[category]/YYYYMMDD-slug.md
     ↓
패턴 감지 + 크로스 레퍼런스 → 검색 가능한 지식 베이스
```

새 프로젝트 적용: `.claude/skills/compound/` 복사 → schema.yaml 커스터마이즈

> ***wrap**: 일 → Skill (자동화) / **compound**: 일 → 인사이트 (지식) / 둘 다 **복리***

6. /compound — 인사이트 축적 스킬

검증된 인사이트를 구조화된 지식으로

/compound 스킬

작동 원리: 세션에서 발견한 인사이트를 **즉시 구조화**

```
"이 방식이 잘 됐다" → /compound 실행
     ↓
1. 도메인/카테고리 자동 분류
2. YAML frontmatter + 본문 생성
3. knowledge/solutions/[category]/YYYYMMDD-slug.md 저장
4. 기존 문서와 크로스 레퍼런스 → 패턴 감지
```

적용법: `.claude/skills/compound/` 복사 → `schema.yaml` 커스터마이즈

누적 효과	설명
1주차	개별 인사이트 문서 5개
1개월	카테고리별 패턴 → **의사결정 기준** 형성
3개월	팀 전체의 **검색 가능한 지식 베이스**

> ***wrap**: 일 → Skill (자동화) / **compound**: 일 → 인사이트 (지식) / 둘 다 **복리***

Agent Teams — 복잡한 작업을 팀으로

복잡한 작업 → **전문가 역할로 분해** → 병렬 실행 → 결과 종합

```
"CI/CD 파이프라인 전체 검토해줘"
          ↓
 #1 architect  (opus)   구조 분석  ┐
 #2 security   (opus)   보안 스캔  ┤ 병렬
 #3 optimizer  (sonnet) 최적화    ┘
 #4 reporter   (sonnet) 리포트    ← 순차 (종합)
          ↓
    종합 리포트
```

모델 선택: "판단 필요?" → **opus** / "기준대로 실행?" → **sonnet**

Claude Code 실험적 기능 — Teammates, 공유 Task List, 에이전트 간 메시지

공식 문서: **https://code.claude.com/docs/en/agent-teams**

7. Skill 만드는 가장 좋은 방법

일을 시키고 → 분석하고 → 스킬로 정제

Skill 개발 워크플로우

```
Step 1: 일을 순차적으로 시킨다
  claude "헬스체크 엔드포인트 만들어줘"
  claude "테스트 작성해줘"
  claude "린트 돌려줘"

Step 2: 내가 했던 작업을 스킬로 만들어달라고 한다
  claude "방금 한 작업을 스킬로 만들어줘"

Step 3: 작업을 구조적으로 분석 → 다이어그램 생성
  → CC가 작업 흐름을 dot/mermaid로 시각화
  → 단계별 입력/출력/의존성 정리

Step 4: 내가 로직을 검토하고 수정
  → 불필요한 단계 제거, 순서 변경, 조건 추가

Step 5: 공식 플러그인으로 SKILL.md 정제
  /plugin-dev:skill-development
  → 표준 형식으로 최종 정리
```

> ***핵심**: 먼저 일을 해보고, 그 경험을 스킬로 **추출**하는 것*

8. 해커톤

직접 만들기 — more token, more wins

스타터 스킬 패키지

Clarify 렌즈

`/vague` `/unknown` `/metamedium`

Core Workflow

`/clarify` `/plan` `/implement` `/wrap` `/worktree`

Advanced

`/compound` `/team-assemble`

해커톤 규칙

1. clarify부터 — 바로 코딩 금지. `/vague` 또는 `/unknown`
2. 결과물 필수 — Skill 1개, PR 1개, CLAUDE.md, 뭐든 OK
3. more token, more wins — 아끼지 말고 CC를 최대한 활용

해커톤 시작

"CC로 해결하고 싶은 가장 큰 pain point 하나"

접근법 제안 → 바로 시작